

Experiment No. 8

Application of Optical Flow for Real-Time Object Tracking and Motion Analysis

Aim:

To implement an optical flow-based motion analysis system for real-time object tracking and evaluate its effectiveness in detecting and tracking moving objects in video sequences using Python and OpenCV.

Environment: Google Colab / Jupyter Notebook

Libraries: OpenCV, NumPy, Matplotlib

Purpose

Optical flow estimates motion between consecutive video frames and can be applied to object tracking, traffic monitoring, human activity recognition, autonomous navigation, video surveillance, and sports analytics.

In this experiment:

- Shi-Tomasi Corner Detection is used to select trackable feature points.
- Lucas-Kanade Sparse Optical Flow tracks those points.
- Motion trajectories, displacement, direction, and average speed are estimated.
- Farneback Dense Optical Flow is used for comparison.
- Effects of speed, illumination, occlusion, and camera motion are discussed.
- Optical-flow tracking is compared conceptually with frame-by-frame object detection.

```
# Step 1: Install/import required libraries
!pip -q install opencv-python-headless matplotlib numpy

import cv2
import numpy as np
import matplotlib.pyplot as plt
import urllib.request
import os
import time
import math

print("OpenCV version:", cv2.__version__)

OpenCV version: 5.0.0
```

Step 1 — Load a real-world video sequence

```
# Download a sample video from the OpenCV repository
video_url =
"https://raw.githubusercontent.com/opencv/opencv/master/samples/data/
vtest.avi"
video_path = "/content/vtest.avi"

try:
    urllib.request.urlretrieve(video_url, video_path)
    print("Sample video downloaded:", video_path)
except Exception as e:
    print("Sample video download failed:", e)
    print("Use the optional upload cell below to provide your own
video.")

Sample video downloaded: /content/vtest.avi

# Optional: upload your own video in Google Colab
# Uncomment and run this cell if you want to use your own video.

# from google.colab import files
# uploaded = files.upload()
# video_path = "/content/" + next(iter(uploaded))
# print("Using:", video_path)

cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    raise RuntimeError("Could not open the video. Upload another video
and set video_path.")

fps = cap.get(cv2.CAP_PROP_FPS)
frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
duration = frame_count / fps if fps and fps > 0 else 0

print(f"Resolution: {width} x {height}")
print(f"FPS: {fps:.2f}")
print(f"Frames: {frame_count}")
print(f"Duration: {duration:.2f} seconds")

cap.release()

Resolution: 768 x 576
FPS: 10.00
Frames: 795
Duration: 79.50 seconds
```

Step 2 — Select feature points using Shi-Tomasi Corner Detection

```
cap = cv2.VideoCapture(video_path)
ret, first_frame = cap.read()

if not ret:
    cap.release()
    raise RuntimeError("Unable to read the first frame.")

first_rgb = cv2.cvtColor(first_frame, cv2.COLOR_BGR2RGB)
first_gray = cv2.cvtColor(first_frame, cv2.COLOR_BGR2GRAY)

# Shi-Tomasi parameters
feature_params = dict(
    maxCorners=150,
    qualityLevel=0.01,
    minDistance=7,
    blockSize=7
)

p0 = cv2.goodFeaturesToTrack(first_gray, mask=None, **feature_params)

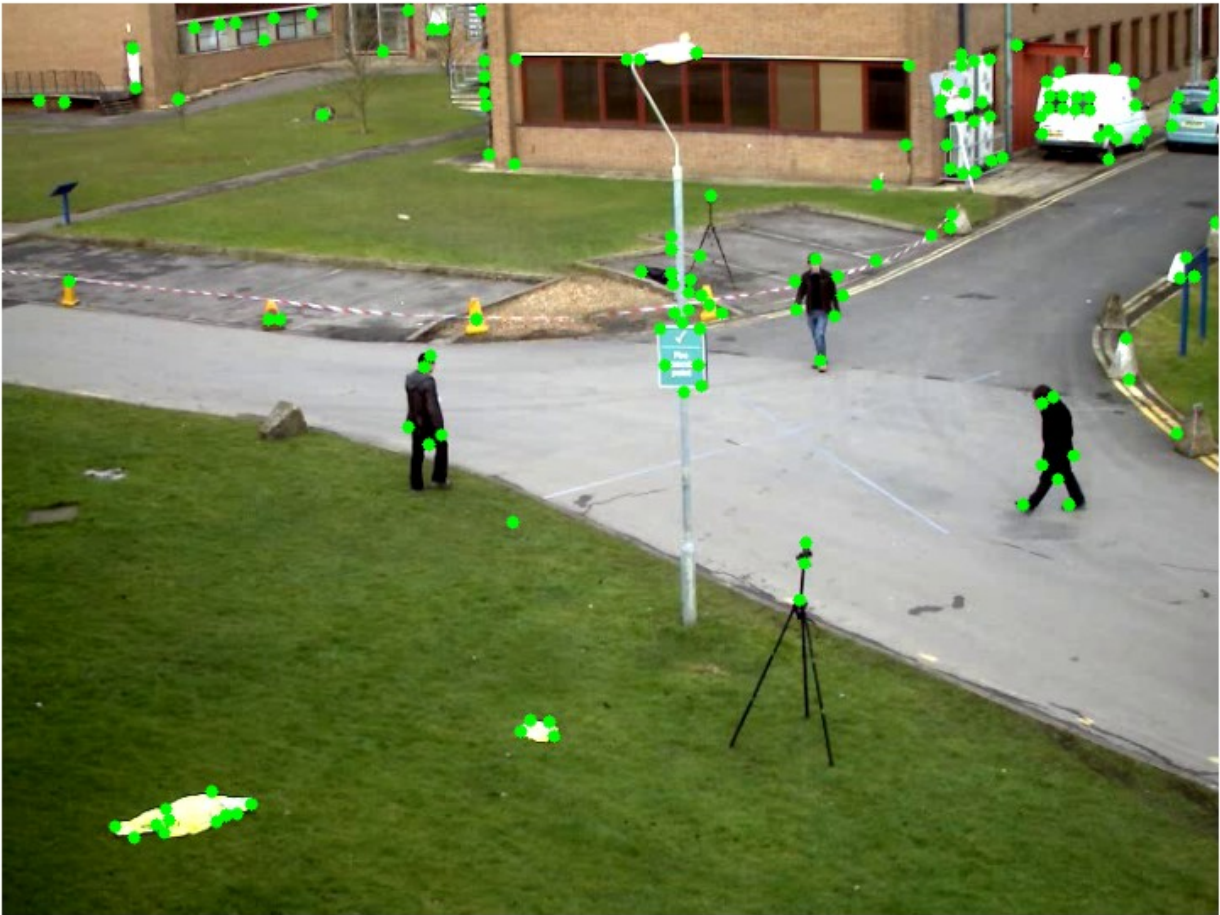
if p0 is None:
    raise RuntimeError("No Shi-Tomasi features detected.")

feature_view = first_frame.copy()
for p in p0:
    x, y = p.ravel()
    cv2.circle(feature_view, (int(x), int(y)), 4, (0, 255, 0), -1)

plt.figure(figsize=(12, 7))
plt.imshow(cv2.cvtColor(feature_view, cv2.COLOR_BGR2RGB))
plt.title(f"Shi-Tomasi Feature Points: {len(p0)} points")
plt.axis("off")
plt.show()

cap.release()
```

Shi-Tomasi Feature Points: 150 points



Step 3–5 — Lucas-Kanade Optical Flow Tracking

Lucas-Kanade estimates the displacement of selected feature points between consecutive frames.

For every tracked point, the experiment calculates:

- displacement (dx , dy)
- magnitude of motion
- direction in degrees
- trajectory across multiple frames

```
# Lucas-Kanade parameters
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10,
0.03)
)
```

```

cap = cv2.VideoCapture(video_path)
ret, old_frame = cap.read()
if not ret:
    cap.release()
    raise RuntimeError("Could not read video.")

old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)

trajectory_mask = np.zeros_like(old_frame)
motion_magnitudes = []
motion_directions = []
frame_index = 0

# Store a subset of trajectories for visualization
trajectory_history = [p0.reshape(-1, 2).copy()]

max_frames = 100 # Keep Colab execution reasonably fast

while frame_index < max_frames:
    ret, frame = cap.read()
    if not ret:
        break

    frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    if p0 is None or len(p0) == 0:
        break

    p1, st, err = cv2.calcOpticalFlowPyrLK(old_gray, frame_gray, p0,
None, **lk_params)

    if p1 is None:
        break

    good_new = p1[st == 1]
    good_old = p0[st == 1]

    frame_magnitudes = []
    frame_directions = []

    for new, old in zip(good_new, good_old):
        a, b = new.ravel()
        c, d = old.ravel()

        dx = a - c
        dy = b - d
        magnitude = float(np.sqrt(dx * dx + dy * dy))
        direction = float(np.degrees(np.arctan2(-dy, dx)) % 360)

```

```

        frame_magnitudes.append(magnitude)
        frame_directions.append(direction)

        # Draw trajectory and displacement vector
        trajectory_mask = cv2.line(
            trajectory_mask, (int(a), int(b)), (int(c), int(d)),
            (0, 255, 0), 2
        )
        frame = cv2.circle(frame, (int(a), int(b)), 3, (0, 0, 255), -
1)

    if frame_magnitudes:
        motion_magnitudes.extend(frame_magnitudes)
        motion_directions.extend(frame_directions)

    img = cv2.add(frame, trajectory_mask)

    # Display every 10th processed frame
    if frame_index % 10 == 0:
        display_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        plt.figure(figsize=(12, 7))
        plt.imshow(display_rgb)
        plt.title(f"Lucas-Kanade Tracking – Frame {frame_index}")
        plt.axis("off")
        plt.show()

    # Keep trajectories for selected frames
    if frame_index % 10 == 0:
        trajectory_history.append(good_new.copy())

    old_gray = frame_gray.copy()
    p0 = good_new.reshape(-1, 1, 2)
    frame_index += 1

cap.release()

print("Processed frames:", frame_index)
print("Tracked motion samples:", len(motion_magnitudes))

```

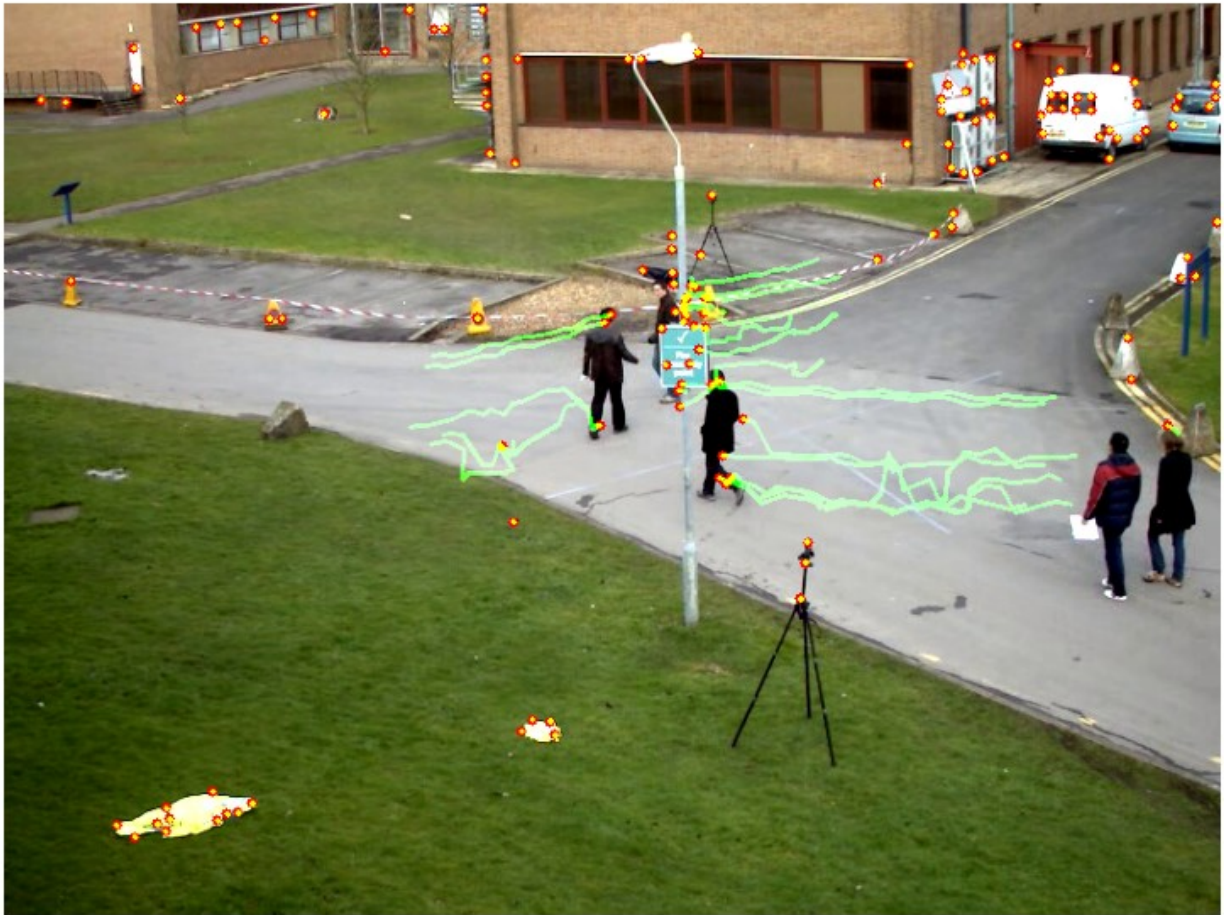

Lucas-Kanade Tracking — Frame 0



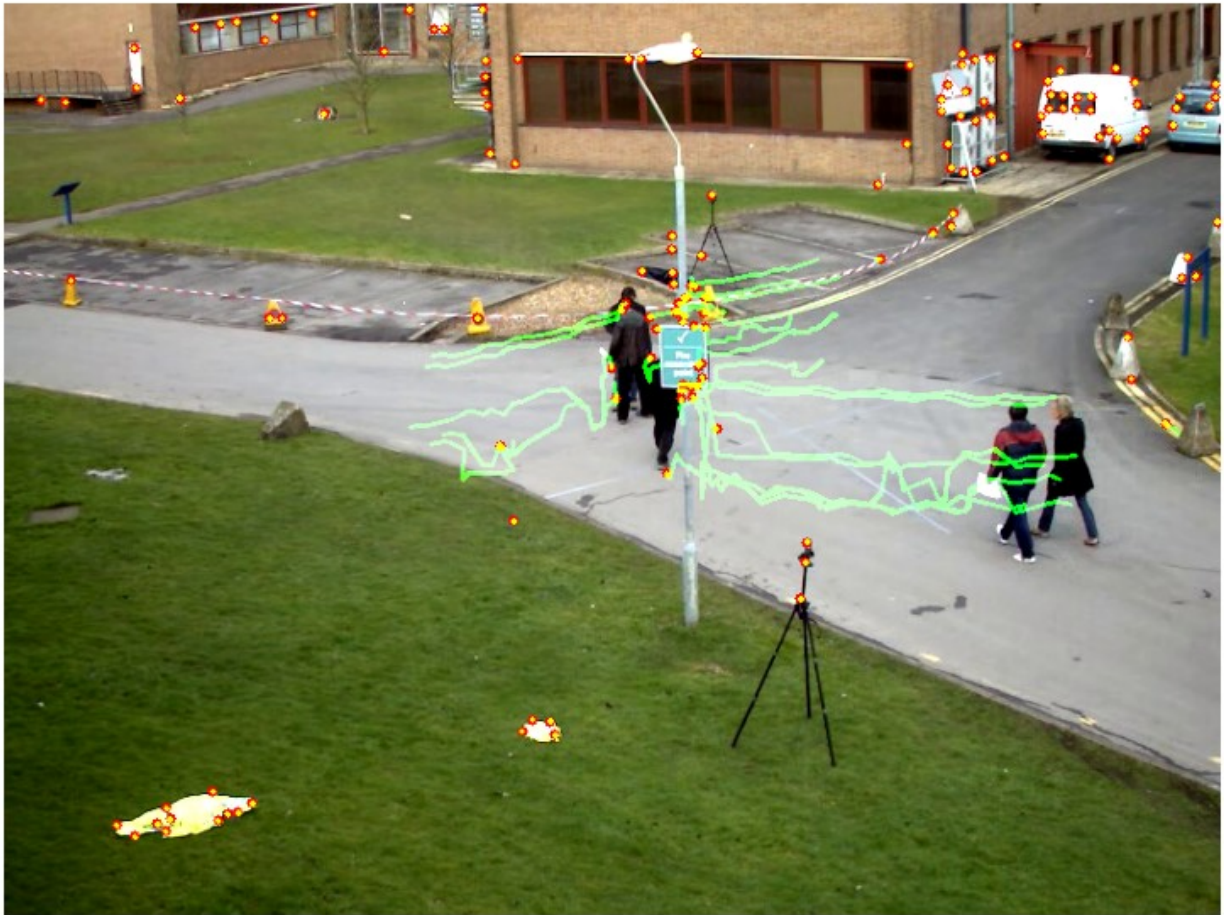
Lucas-Kanade Tracking — Frame 10



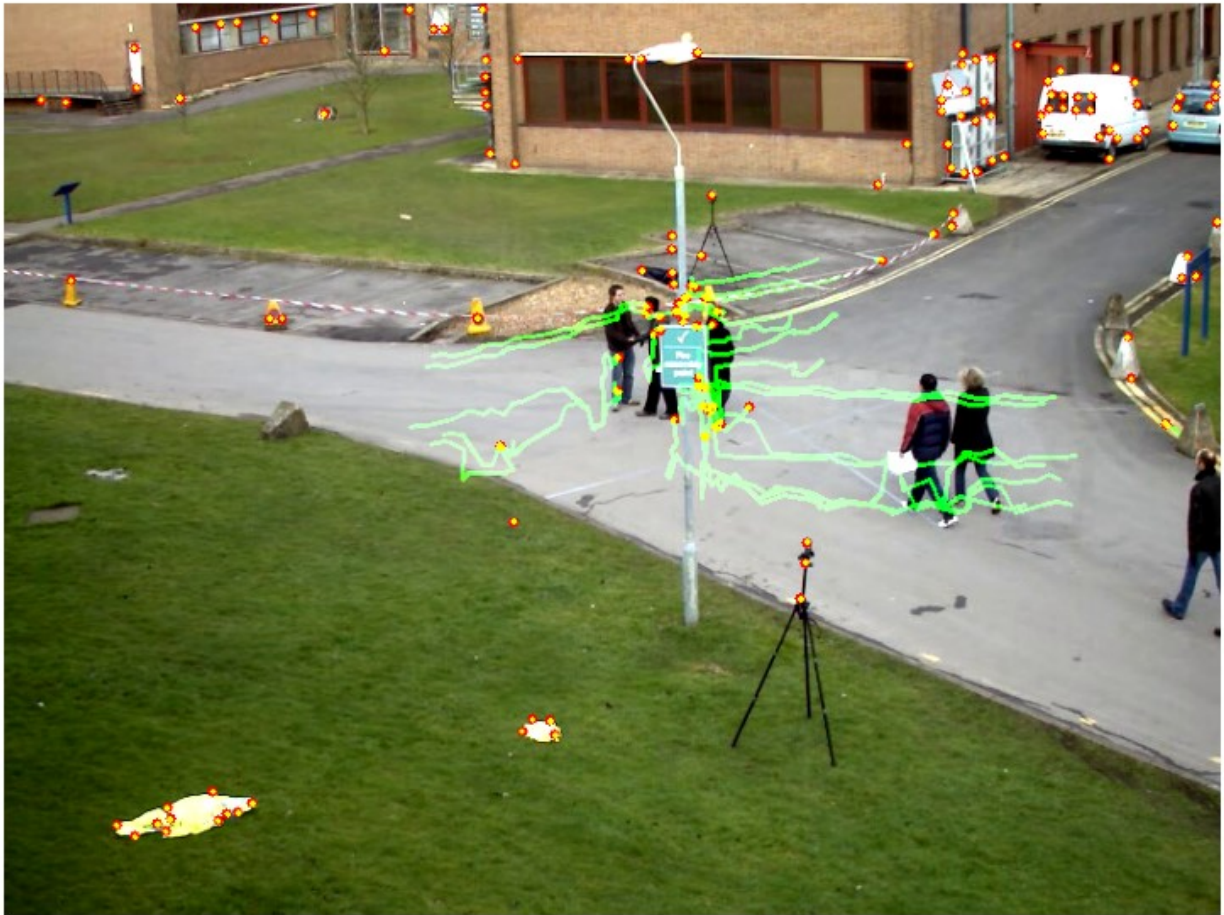
Lucas-Kanade Tracking — Frame 20



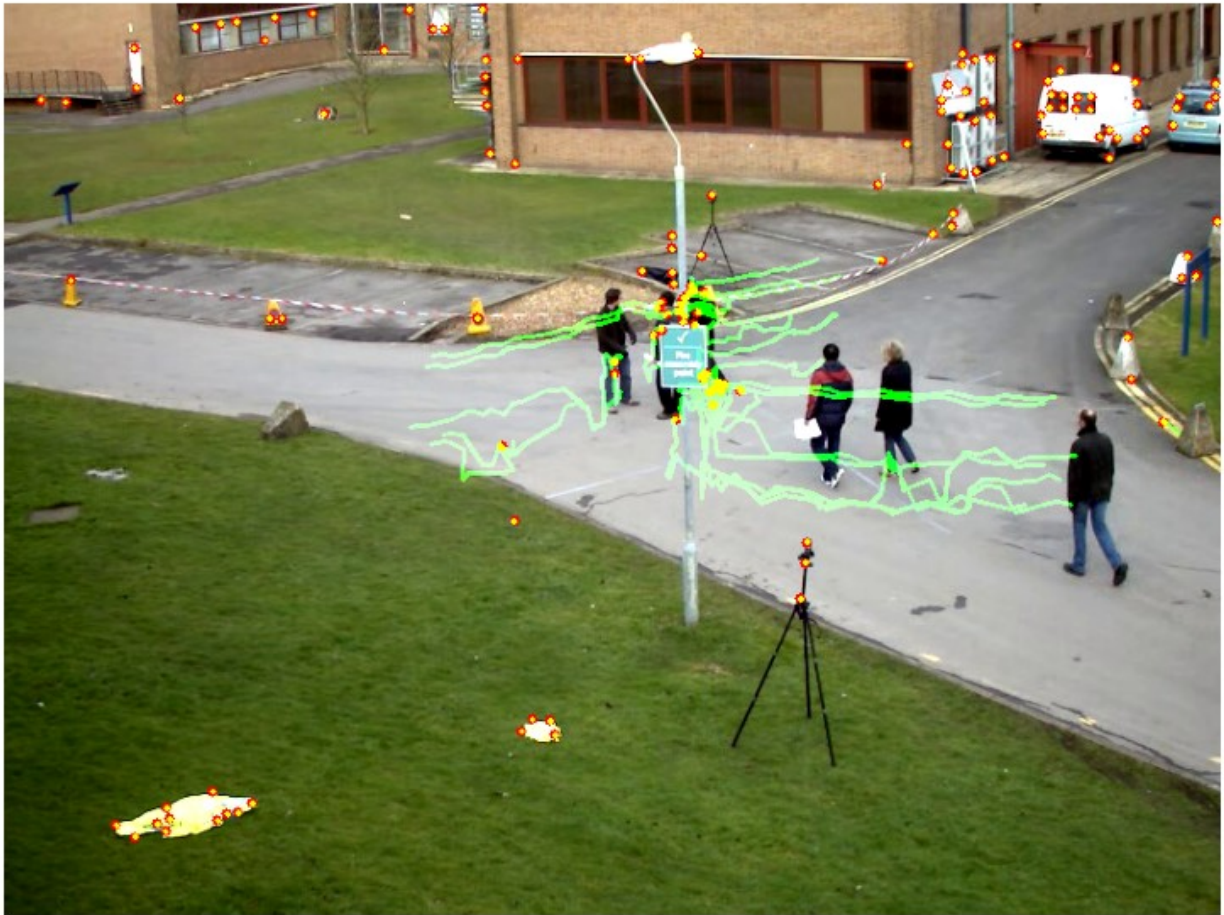
Lucas-Kanade Tracking — Frame 30



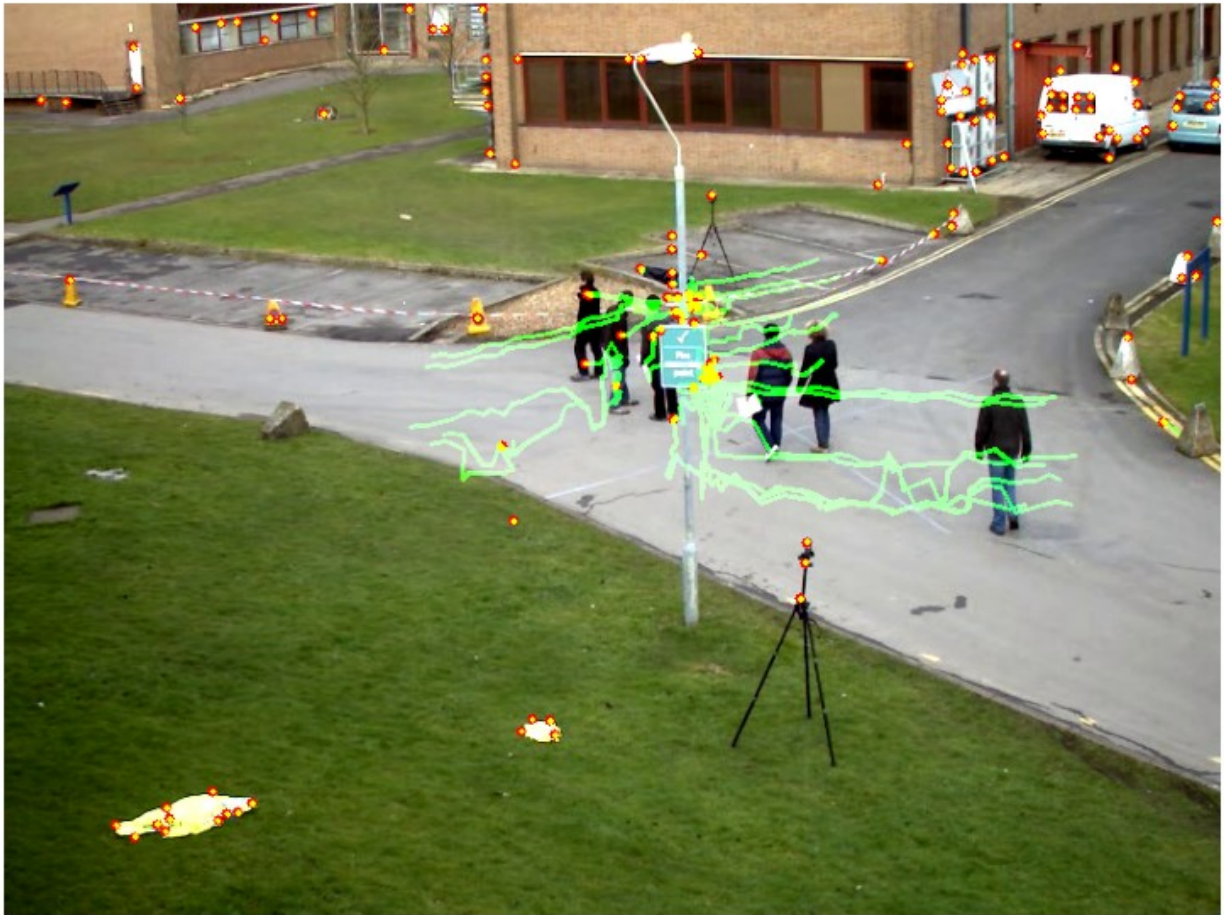
Lucas-Kanade Tracking — Frame 40



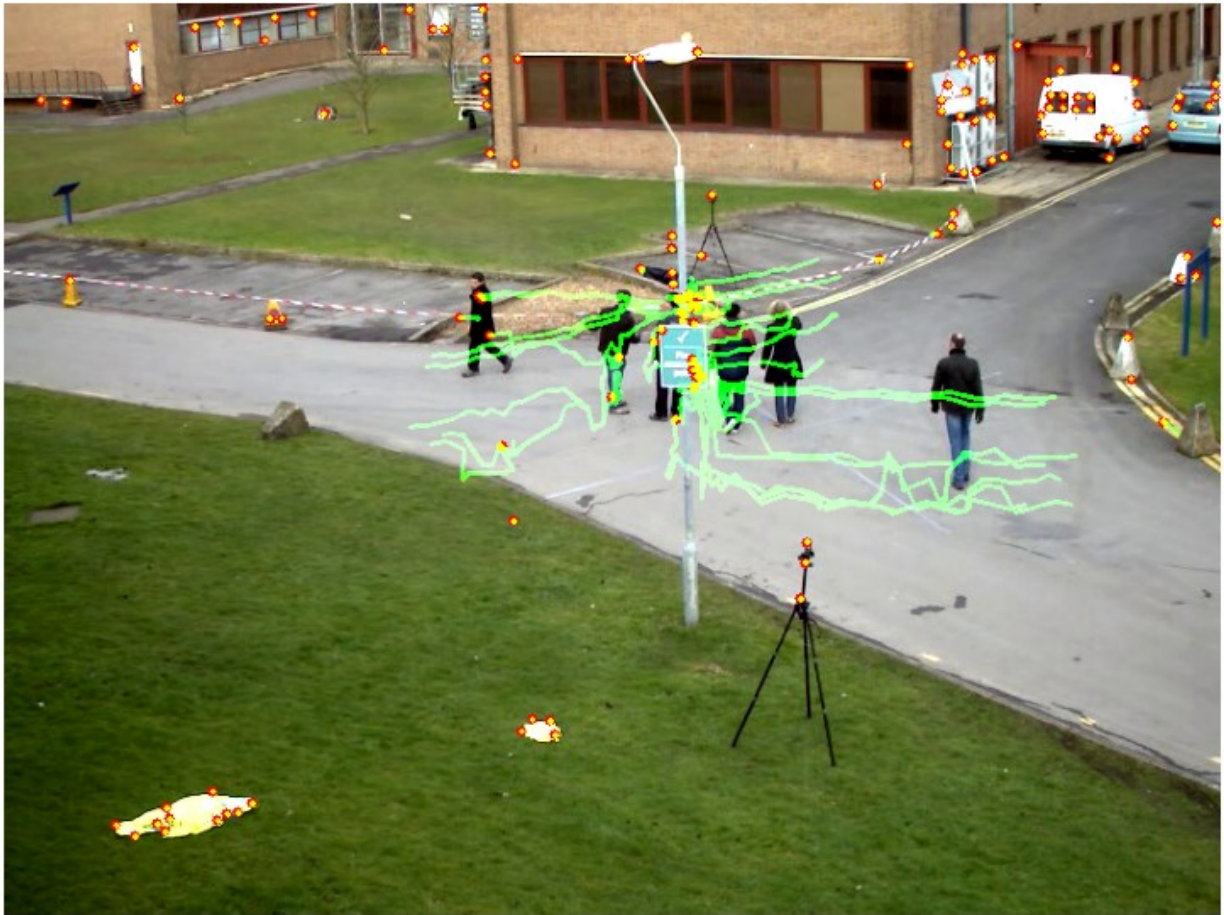
Lucas-Kanade Tracking — Frame 50



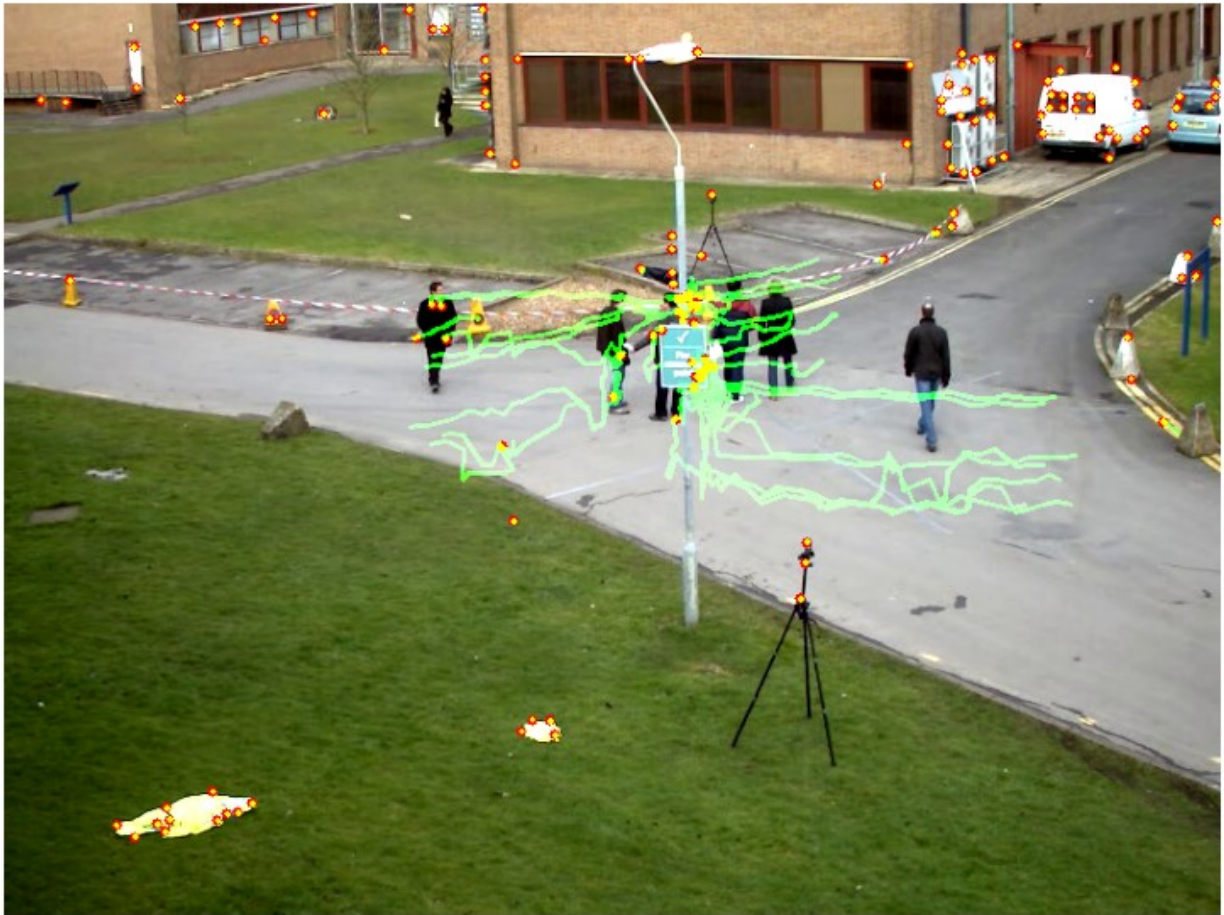
Lucas-Kanade Tracking — Frame 60



Lucas-Kanade Tracking — Frame 70



Lucas-Kanade Tracking — Frame 80



Lucas-Kanade Tracking — Frame 90



Processed frames: 100
Tracked motion samples: 15000

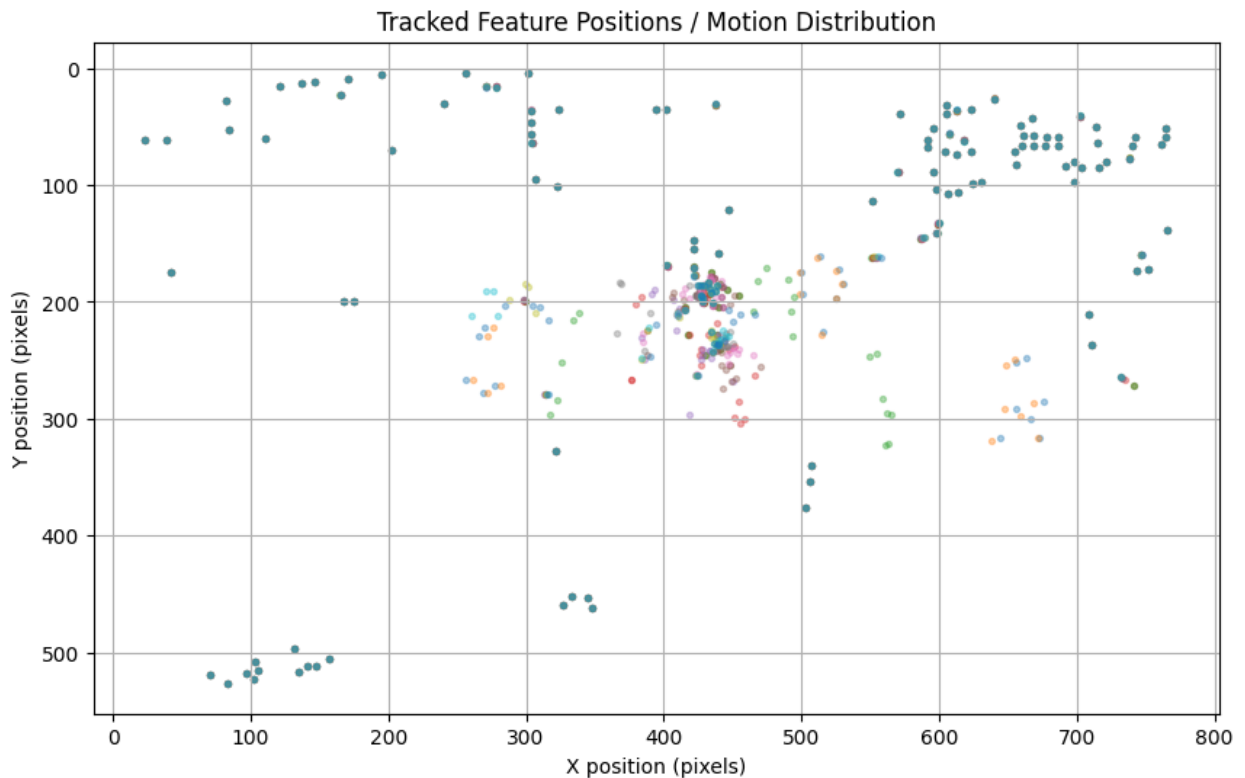
Step 4 — Plot motion trajectory

```
# Visualize feature trajectories on a blank coordinate system.
plt.figure(figsize=(10, 6))

for trajectory in trajectory_history[:]:
    if len(trajectory) == 0:
        continue
    # This is a snapshot of positions rather than a persistent ID-
    # linked trajectory.
    plt.scatter(trajectory[:, 0], trajectory[:, 1], s=8, alpha=0.35)

plt.gca().invert_yaxis()
plt.xlabel("X position (pixels)")
plt.ylabel("Y position (pixels)")
plt.title("Tracked Feature Positions / Motion Distribution")
```

```
plt.grid(True)
plt.show()
```



Step 5 — Calculate magnitude, direction, displacement, and average speed

```
if motion_magnitudes:
    avg_displacement_per_frame = float(np.mean(motion_magnitudes))
    max_displacement = float(np.max(motion_magnitudes))
    avg_speed_px_per_sec = avg_displacement_per_frame * fps

    # Circular mean direction is more appropriate than arithmetic mean
for angles.
    angles = np.radians(motion_directions)
    mean_angle = np.degrees(
        np.arctan2(np.mean(np.sin(angles)), np.mean(np.cos(angles)))
    ) % 360

    print(f"Average feature displacement per frame:
    {avg_displacement_per_frame:.3f} pixels")
    print(f"Maximum feature displacement: {max_displacement:.3f}
    pixels")
    print(f"Estimated average feature speed:
    {avg_speed_px_per_sec:.3f} pixels/second")
```

```
    print(f"Mean motion direction: {mean_angle:.2f} degrees")
else:
    print("No motion samples available.")
```

Average feature displacement per frame: 0.429 pixels
Maximum feature displacement: 39.947 pixels
Estimated average feature speed: 4.287 pixels/second
Mean motion direction: 119.66 degrees

Step 6 — Farneback Dense Optical Flow

```
# Farneback dense optical flow
cap = cv2.VideoCapture(video_path)
ret, prev_frame = cap.read()

if not ret:
    cap.release()
    raise RuntimeError("Could not read video.")

prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)
dense_magnitudes = []
dense_times = []

for i in range(30):
    ret, frame = cap.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    start = time.perf_counter()
    flow = cv2.calcOpticalFlowFarneback(
        prev_gray, gray, None,
        pyr_scale=0.5,
        levels=3,
        winsize=15,
        iterations=3,
        poly_n=5,
        poly_sigma=1.2,
        flags=0
    )
    dense_times.append(time.perf_counter() - start)

    mag, ang = cv2.cartToPolar(flow[..., 0], flow[..., 1])
    dense_magnitudes.append(float(np.mean(mag)))

# HSV representation of motion
hsv = np.zeros_like(cv2.cvtColor(frame, cv2.COLOR_BGR2HSV))
hsv[..., 1] = 255
```



```

hsv[..., 0] = (ang * 180 / np.pi / 2).astype(np.uint8)
hsv[..., 2] = cv2.normalize(mag, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)

if i in [0, 10, 20]:
    dense_rgb = cv2.cvtColor(cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR),
cv2.COLOR_BGR2RGB)
    plt.figure(figsize=(12, 7))
    plt.imshow(dense_rgb)
    plt.title(f"Farneback Dense Optical Flow — Frame {i}")
    plt.axis("off")
    plt.show()

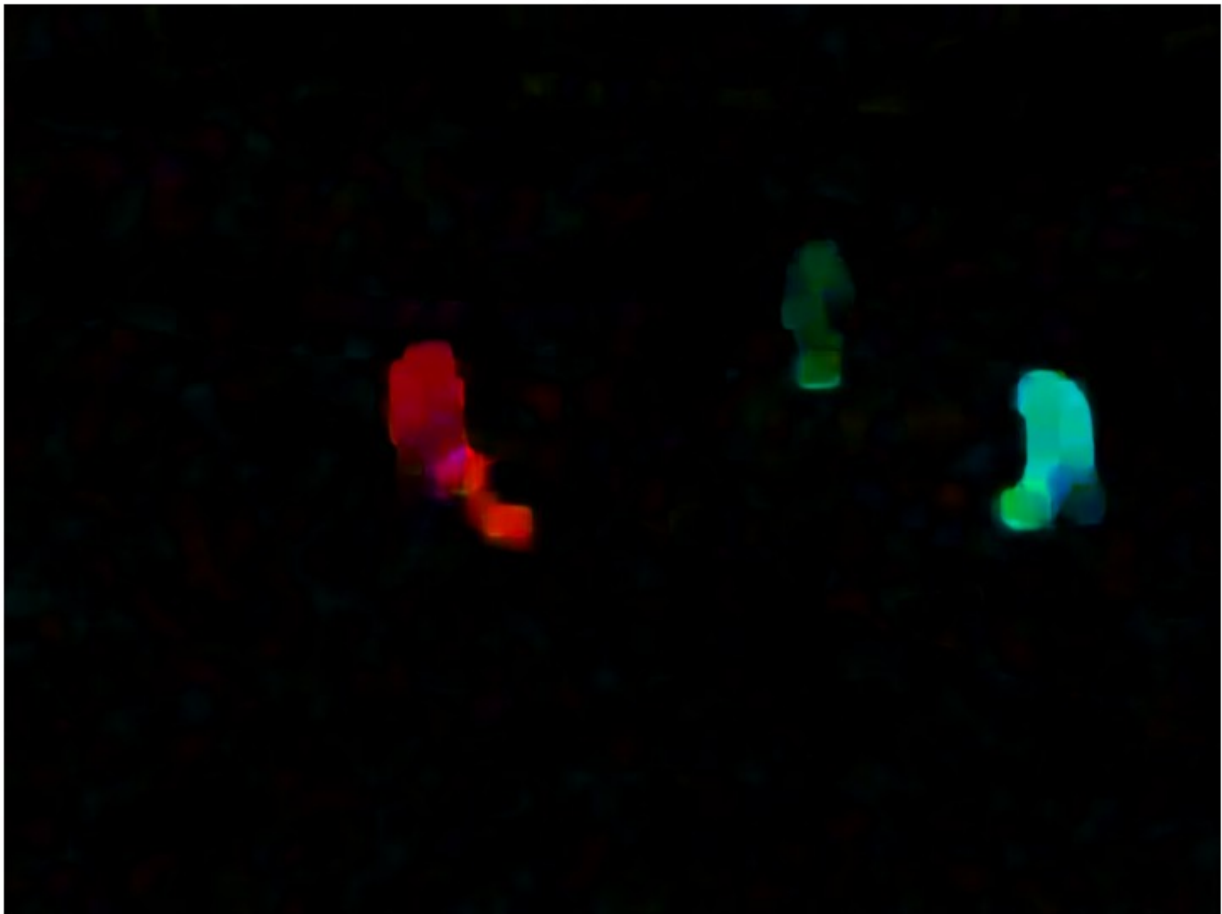
prev_gray = gray

cap.release()

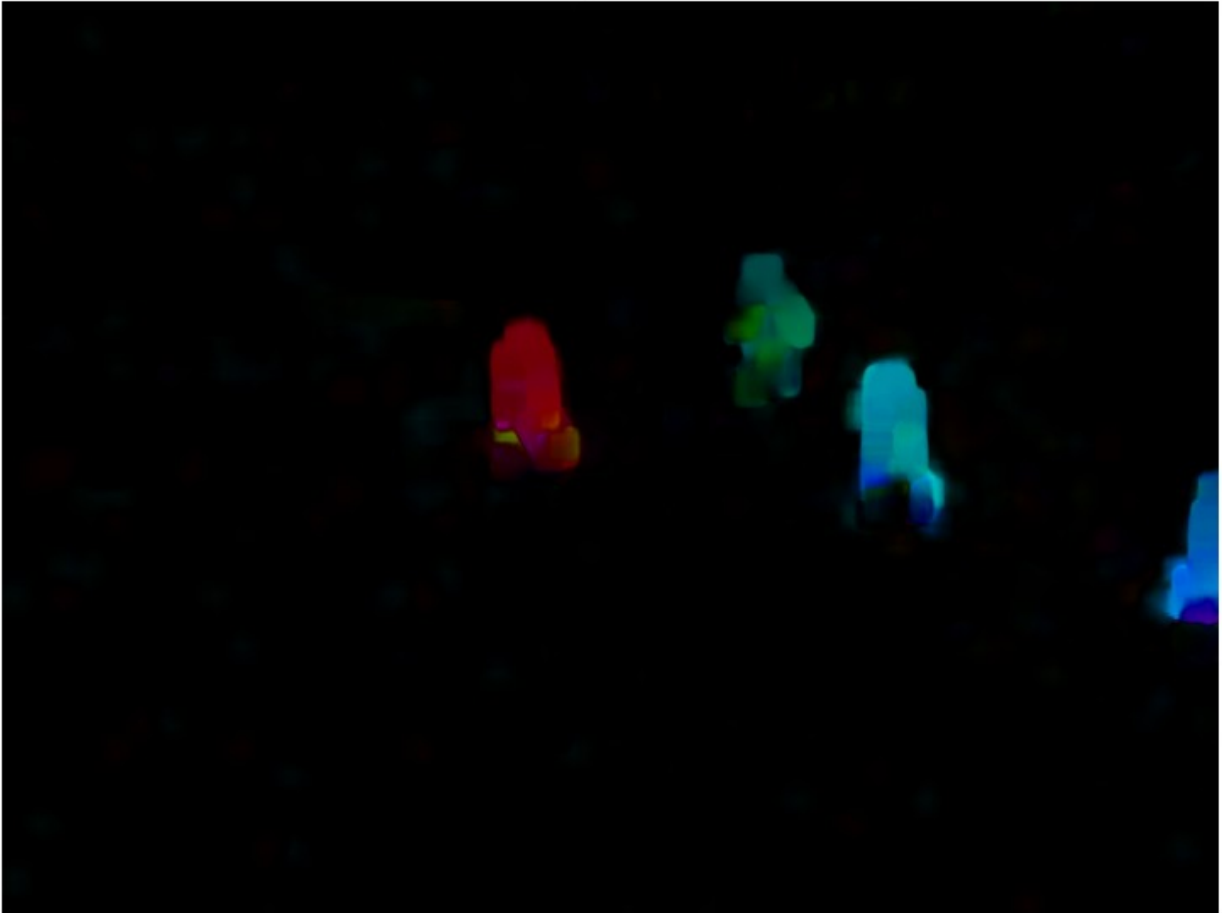
print("Farneback frames processed:", len(dense_magnitudes))
print("Average dense motion magnitude:", np.mean(dense_magnitudes) if
dense_magnitudes else 0)

```

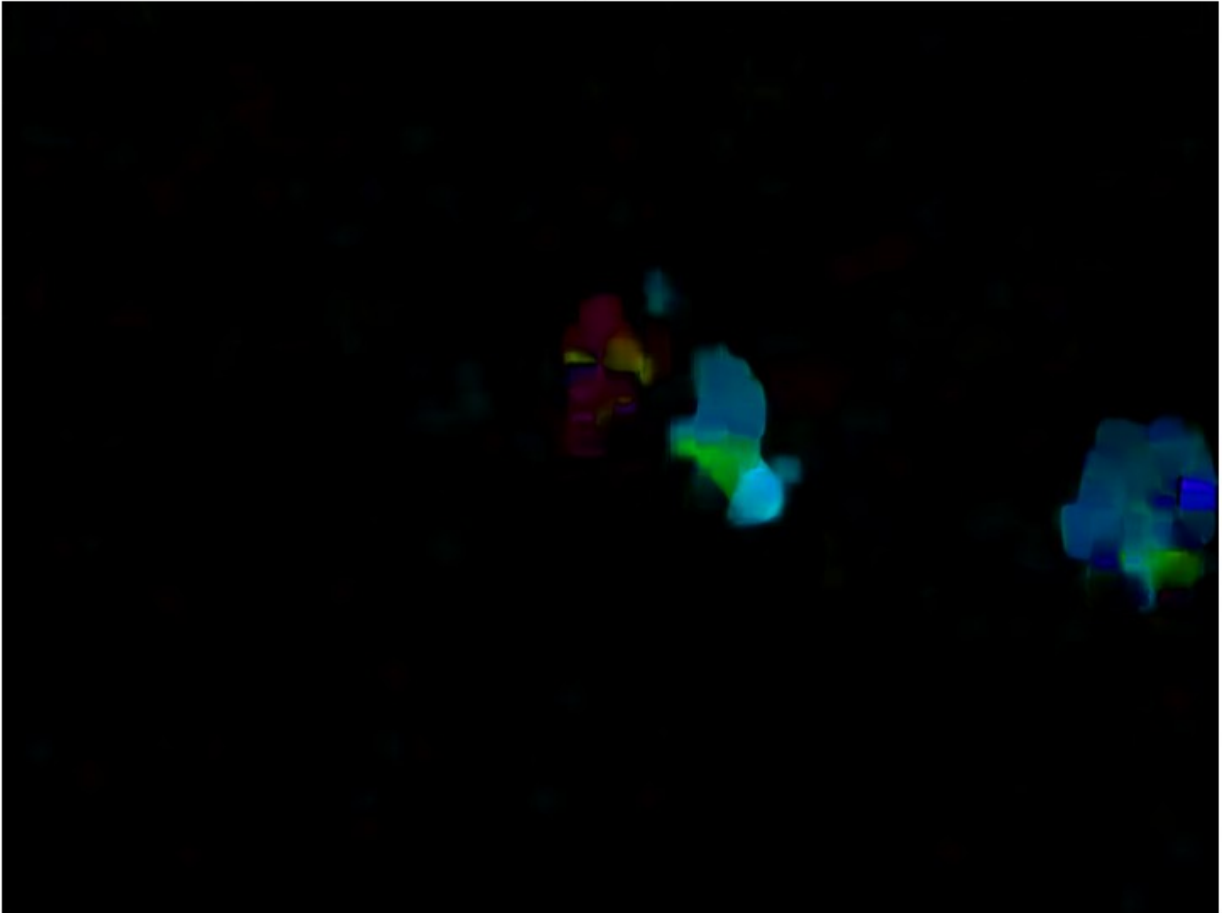
Farneback Dense Optical Flow — Frame 0



Farneback Dense Optical Flow — Frame 10



Farneback Dense Optical Flow — Frame 20



```
Farneback frames processed: 30  
Average dense motion magnitude: 0.3122419094045957
```

Step 7 — Analyze practical tracking challenges

Object speed

Fast motion can cause larger frame-to-frame displacement, making feature correspondence harder and increasing tracking errors.

Illumination changes

Sudden brightness or contrast changes can alter local image appearance and reduce correspondence quality.

Occlusion

When an object or feature becomes hidden, its tracked points may disappear. New Shi-Tomasi features can be detected to recover tracking.

Camera movement

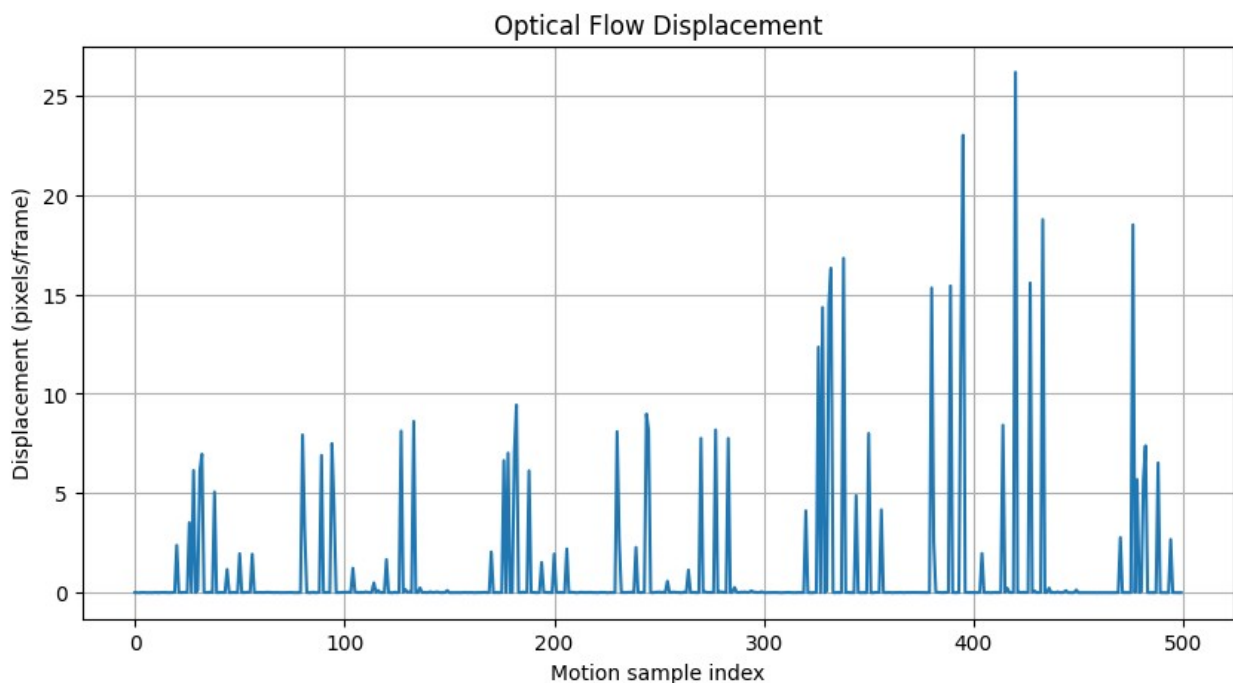
Camera motion creates optical flow across much of the image, so the measured flow contains both object motion and camera-induced motion. Background motion compensation or a stationary reference is often needed.

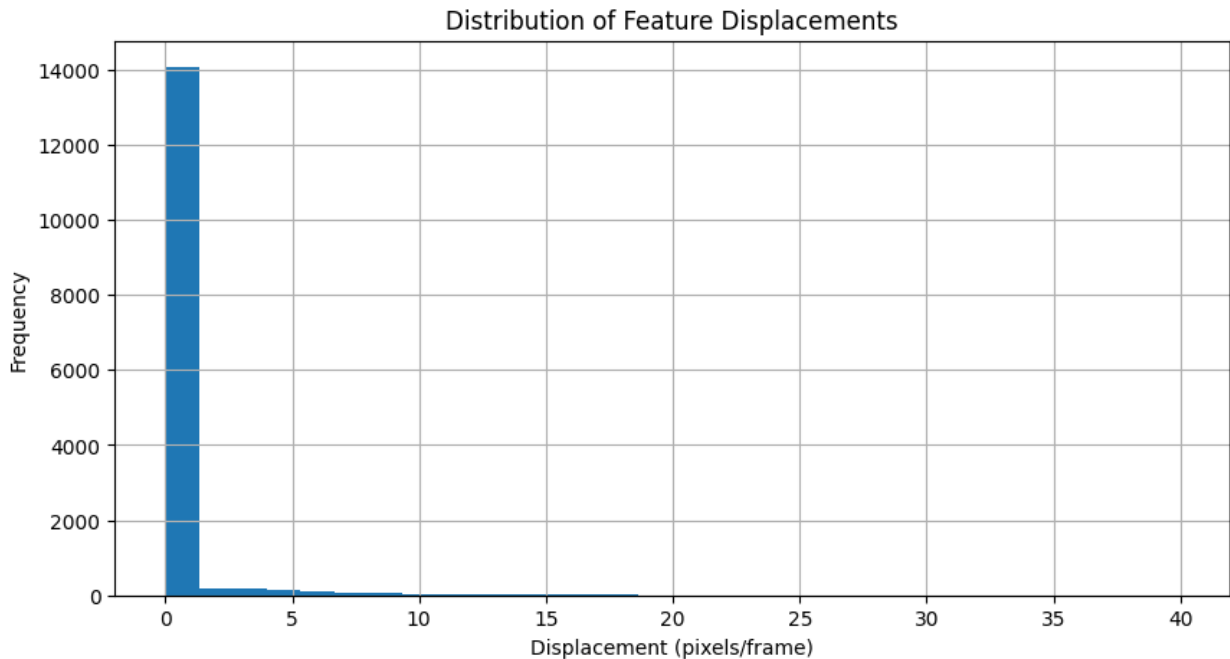
Step 8 — Estimate trajectory and average speed

```
# Summary plots for the measured motion
if motion_magnitudes:
    sample = motion_magnitudes[:500]

    plt.figure(figsize=(10, 5))
    plt.plot(sample)
    plt.xlabel("Motion sample index")
    plt.ylabel("Displacement (pixels/frame)")
    plt.title("Optical Flow Displacement")
    plt.grid(True)
    plt.show()

    plt.figure(figsize=(10, 5))
    plt.hist(motion_magnitudes, bins=30)
    plt.xlabel("Displacement (pixels/frame)")
    plt.ylabel("Frequency")
    plt.title("Distribution of Feature Displacements")
    plt.grid(True)
    plt.show()
```





Step 9 — Compare Optical Flow with Conventional Frame-by-Frame Detection

```
comparison = {
    "Optical Flow": [
        "Tracks motion/feature displacement",
        "Does not require an object class label",
        "Works directly from consecutive frames",
        "Can be computationally efficient for sparse tracking",
        "Can struggle with occlusion and large appearance changes"
    ],
    "Frame-by-frame Detection": [
        "Detects objects independently in each frame",
        "Can identify semantic classes such as person/car",
        "Can recover after temporary tracking loss",
        "Usually requires a trained detector for semantic objects",
        "Often more computationally expensive"
    ]
}

for method, points in comparison.items():
    print(f"\n{method}")
    print("-" * len(method))
    for p in points:
        print("•", p)
```

Optical Flow

- Tracks motion/feature displacement
- Does not require an object class label
- Works directly from consecutive frames
- Can be computationally efficient for sparse tracking
- Can struggle with occlusion and large appearance changes

Frame-by-frame Detection

- Detects objects independently in each frame
- Can identify semantic classes such as person/car
- Can recover after temporary tracking loss
- Usually requires a trained detector for semantic objects
- Often more computationally expensive

Step 9 — Conceptual accuracy/performance comparison

*# A simple timing comparison on the same short video segment.
This compares the computational time of Lucas-Kanade and Farneback,
not their detection accuracy.*

```
def benchmark_lk(path, n_frames=30):
    cap = cv2.VideoCapture(path)
    ret, frame = cap.read()
    if not ret:
        cap.release()
        return None

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    pts = cv2.goodFeaturesToTrack(gray, mask=None, **feature_params)
    total = 0.0
    count = 0

    while count < n_frames:
        ret, frame2 = cap.read()
        if not ret:
            break
        gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

        t0 = time.perf_counter()
        if pts is not None and len(pts):
            cv2.calcOpticalFlowPyrLK(gray, gray2, pts, None,
**lk_params)
            total += time.perf_counter() - t0

        gray = gray2
        count += 1

    cap.release()
```

```

    return total / max(count, 1)

def benchmark_farneback(path, n_frames=30):
    cap = cv2.VideoCapture(path)
    ret, frame = cap.read()
    if not ret:
        cap.release()
        return None

    prev = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    total = 0.0
    count = 0

    while count < n_frames:
        ret, frame2 = cap.read()
        if not ret:
            break
        curr = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

        t0 = time.perf_counter()
        cv2.calcOpticalFlowFarneback(
            prev, curr, None, 0.5, 3, 15, 3, 5, 1.2, 0
        )
        total += time.perf_counter() - t0

        prev = curr
        count += 1

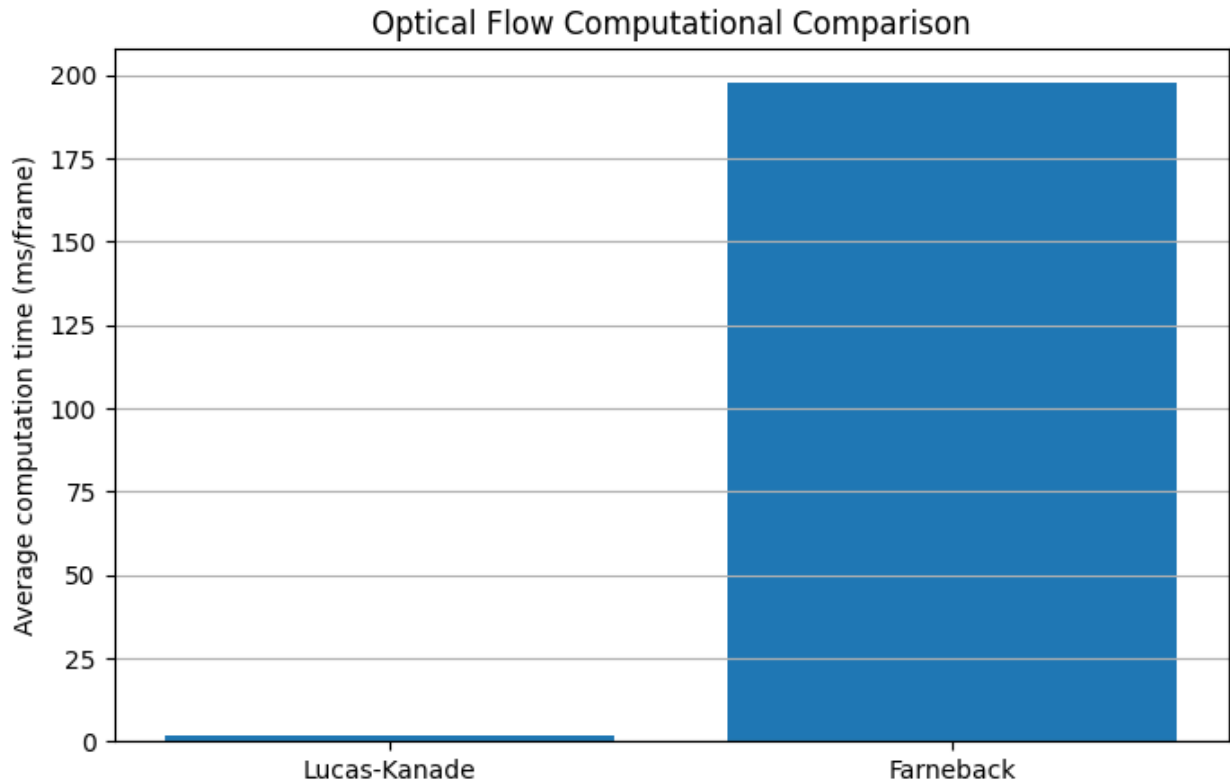
    cap.release()
    return total / max(count, 1)

lk_time = benchmark_lk(video_path)
fb_time = benchmark_farneback(video_path)

if lk_time is not None and fb_time is not None:
    plt.figure(figsize=(8, 5))
    plt.bar(["Lucas-Kanade", "Farneback"], [lk_time * 1000, fb_time *
1000])
    plt.ylabel("Average computation time (ms/frame)")
    plt.title("Optical Flow Computational Comparison")
    plt.grid(axis="y")
    plt.show()

    print(f"Lucas-Kanade: {lk_time*1000:.3f} ms/frame")
    print(f"Farneback:      {fb_time*1000:.3f} ms/frame")

```



Lucas-Kanade: 1.880 ms/frame
Farneback: 197.945 ms/frame

Step 10 — Observations and Practical Applications

Observations

1. Lucas-Kanade tracks selected feature points and is generally efficient for sparse motion tracking.
2. Shi-Tomasi provides strong corner features that are useful for correspondence.
3. Farneback estimates motion densely over the image and provides richer motion information.
4. Fast motion, illumination variation, occlusion, and camera movement can reduce tracking reliability.
5. Camera motion can produce widespread flow that must be separated from independently moving objects.
6. Optical-flow speed estimates are in pixels/second unless camera calibration and scene geometry are available.

Practical applications

- Traffic monitoring and vehicle tracking
- Autonomous driving and navigation
- Video surveillance

- Human activity recognition
- Sports analytics
- Robotics
- Crowd/motion analysis

Questions and Answers

1. How does object tracking differ from object detection?

Object detection identifies objects in individual frames, usually returning their locations and classes. Object tracking maintains the identity and location of an object across multiple frames.

2. Explain how Optical Flow can be used for real-time object tracking.

Optical Flow estimates the displacement of pixels or feature points between consecutive frames. By following these points over time, the system can estimate an object's position, trajectory, direction, and speed without running a full detector on every frame.

3. What is the role of Shi-Tomasi Corner Detection in the Lucas-Kanade Optical Flow algorithm?

Shi-Tomasi selects strong, distinctive corner points with sufficient local intensity variation. Lucas-Kanade then tracks these points between frames, making the tracking more reliable than using arbitrary pixels.

4. Why is Optical Flow suitable for motion analysis in videos?

Video contains temporal information, and Optical Flow directly estimates apparent motion between consecutive frames. It can therefore reveal direction, displacement, speed, trajectories, and regions of movement.

5. What challenges arise while tracking fast-moving or partially occluded objects?

Fast motion can create large displacements and correspondence errors. Occlusion can make tracked features disappear. Motion blur, low texture, illumination changes, and abrupt camera movement can further reduce tracking accuracy.

6. Compare Optical Flow-based tracking with deep learning-based object tracking methods.

Optical Flow is generally lightweight and does not require object-class training. Deep-learning trackers can provide semantic object identities and are usually more robust to appearance changes, but they require trained models and more computational resources. Hybrid systems often combine detection with optical-flow tracking.

7. How can Optical Flow be used in traffic monitoring and autonomous driving systems?

It can estimate vehicle motion, lane movement, relative motion, road scene dynamics, and the motion of pedestrians or other road users. In autonomous driving, flow can also contribute to visual odometry, obstacle motion estimation, and ego-motion analysis.

8. Explain the effect of camera motion on Optical Flow estimation.

Camera movement causes apparent motion in the background, so Optical Flow may report motion even for stationary objects. Camera motion can therefore mask object motion unless global motion is estimated and compensated.

9. Mention five real-world applications where motion analysis using Optical Flow is commonly employed.

1. Traffic and vehicle monitoring
2. Video surveillance
3. Autonomous vehicles
4. Human activity recognition
5. Sports analytics

10. How can Optical Flow improve the performance of surveillance, robotics, and human activity recognition systems?

It provides temporal motion information that can help detect unusual movement, estimate robot/camera motion, track moving targets, and distinguish human actions based on movement patterns. It can also reduce the need to run expensive object detection on every frame.

Conclusion

In this experiment, an optical-flow-based motion tracking system was implemented using OpenCV. Shi-Tomasi Corner Detection was used to obtain trackable features, and Lucas-Kanade Optical Flow was used to estimate their motion across consecutive frames. Motion trajectories, displacement, direction, and average speed were analyzed. Farneback Dense Optical Flow was also implemented to obtain dense motion information and compare computational behavior.

The experiment demonstrates that Optical Flow is a useful foundation for real-time motion analysis and can support intelligent vision applications such as surveillance, traffic monitoring, robotics, autonomous navigation, and human activity recognition.